



Security Assessment Report Exponent Order Book

May 11, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Exponent Order Book.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 24 issues or questions.

Task	Type	Commit
exponent_orderbook	Solana	8c8b2db (Nov 06, 2025)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview	3
Findings in Detail	4
[H-01] Incorrect expired offer removal logic	4
[H-02] Incorrect last_seen_sy_index initialization inflates interest	6
[H-03] Missing prev_offer_idx reset causes cross-price linkage	8
[H-04] User escrow slot exhaustion	11
[M-01] Incorrect maker taker fee fields	13
[M-02] Rounding error in SY to PT/YT strip calculation	15
[M-03] Crossed book risk caused by post_offer match limit	18
[M-04] Desynchronized SY exchange rates cause yield loss or DoS	20
[M-05] Invalid escrow lookup breaks offer cancellation	22
[L-01] Missing admin checks in orderbook initialization instructions	24
[L-02] Unsafe f64 to integer casting risks	25
[L-03] Precision loss accumulation causes SY deficit	27
[L-04] SY rounding misguides merge/strip	30
[L-05] Core program Strip/Merge limit blocks virtual-virtual matching	33
[I-01] Missing check for threshold_amount during configuration	37
[I-02] Missing offer expiry time check	38
[I-03] Zero filled amount leads to maker loss and vault SY depletion	39
[I-04] Nearest price rounding violates user-defined limit	42
[I-05] Missing validation for fee_rate_bp	43
[I-06] Orderbook rent exhaustion DoS risk	44
[I-07] DoS the orderbook with no penalty	46
[I-08] Missing explicit owner check in set_configuration_options	47
[Q-01] Missing user distribution for emission rewards	48
[Q-02] Discrepancies between event data and the actual transferred amount	49
Appendix: Methodology and Scope of Work	52

Result Overview

Issue	Impact	Status
EXPONENT_ORDERBOOK		
[H-01] Incorrect expired offer removal logic	High	Resolved
[H-02] Incorrect last_seen_sy_index initialization inflates interest	High	Resolved
[H-03] Missing prev_offer_idx reset causes cross-price linkage	High	Resolved
[H-04] User escrow slot exhaustion	High	Resolved
[M-01] Incorrect maker taker fee fields	Medium	Resolved
[M-02] Rounding error in SY to PT/YT strip calculation	Medium	Resolved
[M-03] Crossed book risk caused by post_offer match limit	Medium	Resolved
[M-04] Desynchronized SY exchange rates cause yield loss or DoS	Medium	Resolved
[M-05] Invalid escrow lookup breaks offer cancellation	Medium	Resolved
[L-01] Missing admin checks in orderbook initialization instructions	Low	Resolved
[L-02] Unsafe f64 to integer casting risks	Low	Resolved
[L-03] Precision loss accumulation causes SY deficit	Low	Resolved
[L-04] SY rounding misguides merge/strip	Low	Resolved
[L-05] Core program Strip/Merge limit blocks virtual-virtual matching	Low	Resolved
[I-01] Missing check for threshold_amount during configuration	Info	Acknowledged
[I-02] Missing offer expiry time check	Info	Resolved
[I-03] Zero filled amount leads to maker loss and vault SY depletion	Info	Resolved
[I-04] Nearest price rounding violates user-defined limit	Info	Resolved
[I-05] Missing validation for fee_rate_bp	Info	Resolved
[I-06] Orderbook rent exhaustion DoS risk	Info	Resolved
[I-07] DoS the orderbook with no penalty	Info	Resolved
[I-08] Missing explicit owner check in set_configuration_options	Info	Resolved
[Q-01] Missing user distribution for emission rewards	Question	Acknowledged
[Q-02] Discrepancies between event data and the actual transferred amount	Question	Resolved

Findings in Detail

EXPONENT_ORDERBOOK

[H-01] Incorrect expired offer removal logic

Identified in commit [8c8b2db](#).

The `remove_expired_offers` function walks every price level to identify expired offers. However, the expiry check is inverted: it enqueues offers where `expiry_at > now` instead of `expiry_at <= now`.

Because `remove_offer` is then called with `withdraw_funds = false`, any administrator who tries to clean stale orders actually strips still-valid liquidity from the book while leaving genuinely expired orders in place indefinitely.

```

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1375 | pub fn remove_expired_offers(&mut self, sy_exchange_rate: Number) -> Result<()> {
1380 |     {
1381 |         let price_iter = &mut self.get_price_iterator(OfferType::BuyYt).peekable();
1382 |         while price_iter.peek().is_some() && removed_counter < MAX_REMOVED_OFFERS {
1389 |             while current_offer_idx != SENTINEL && removed_counter < MAX_REMOVED_OFFERS {
1390 |                 if self.offers.get(current_offer_idx).get_value().expiry_at > now {
1391 |                     offers_to_remove.push((prev_offer_idx, current_offer_idx));
1392 |                     removed_counter += 1;
1393 |                 }
1394 |                 prev_offer_idx = current_offer_idx;
1395 |                 current_offer_idx = self
1396 |                     .offers
1397 |                     .get_register(current_offer_idx, NEXT_OFFER_REGISTER);
1398 |             }
1399 |             while current_offer_idx != SENTINEL && removed_counter < MAX_REMOVED_OFFERS {
1400 |                 if self.offers.get(current_offer_idx).get_value().expiry_at > now {
1401 |                     offers_to_remove.push((prev_offer_idx, current_offer_idx));
1402 |                     removed_counter += 1;
1403 |                 }
1404 |                 prev_offer_idx = current_offer_idx;
1405 |                 current_offer_idx = self
1406 |                     .offers
1407 |                     .get_register(current_offer_idx, NEXT_OFFER_REGISTER);
1408 |             }
1409 |         }
1410 |     }
1411 | }
1412 |
1413 | // reverse to prevent removing previous before current
1414 | offers_to_remove.reverse();
1415 | for removed_offer in &offers_to_remove {
1416 |     self.remove_offer(removed_offer.0, removed_offer.1, sy_exchange_rate, false)?;
1417 | }
1418 | Ok(())
1419 | }

```

As a result, legitimate orders can be forcibly removed, and their escrowed assets returned long before

they expire, while expired offers remain stuck, locking user funds. A bad actor with admin privileges (or an automated maintenance script) can repeatedly call `remove_expired_offers` to drain active liquidity or grief market participants, and users cannot reclaim funds from truly expired orders via this path.

1. Alice submits a long-dated offer that is still valid; its `expiry_at` is weeks in the future.
2. The admin cron job calls `remove_expired_offers`, which mistakenly deletes Alice's valid order, crediting the assets back but removing her liquidity, while expired offers from other users stay lodged and unusable.

It's recommended to fix the comparator so only `expiry_at <= now` entries are collected, ensuring `remove_expired_offers` actually scrubs stale orders. After patching, run a one-time cleanup to free any previously stuck offers.

Current version:

```
if self.offers.get(current_offer_idx).get_value().expiry_at > now {  
    offers_to_remove.push((prev_offer_idx, current_offer_idx));  
}
```

Proposed changes:

```
if self.offers.get(current_offer_idx).get_value().expiry_at <= now {  
    offers_to_remove.push((prev_offer_idx, current_offer_idx));  
}
```

Resolution

This issue has been fixed by [43c8d45](#).

EXPONENT_ORDERBOOK

[H-02] Incorrect last_seen_sy_index initialization inflates interest

Identified in commit [8c8b2db](#).

The orderbook protocol deposits users' **YT** tokens into the **core** program's yield position so that **SY** interest can accumulate. Each user's **SY** interest is tracked by the **stage_yt_interest** function, which is called before **staked_yt_amount** is updated in order to compute the correct incremental interest.

```
/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
165 | pub fn add_balance_on_fill_offer(
172 | ) -> Result<> {
173 |     let user_escrow = self.user_escrows.get_mut(user_escrow_idx).get_value_mut();
174 |     match offer_type {
175 |         OfferType::BuyYt => {
176 |             if virtual_offer {
177 |                 self.financials.sy_fee_balance = self.financials.sy_fee_balance
178 |                     .checked_add(fill_escrow_result.fee_amount).unwrap();
179 |                 user_escrow.sy_amount = user_escrow.sy_amount
180 |                     .checked_add(fill_escrow_result.added_amount).expect("SY overflow");
181 |             } else {
182 |                 self.financials.yt_fee_balance = self.financials.yt_fee_balance
183 |                     .checked_add(fill_escrow_result.fee_amount).unwrap();
184 |                 user_escrow.stage_yt_interest(sy_exchange_rate);
185 |                 user_escrow.staked_yt_amount = user_escrow.staked_yt_amount
186 |                     .checked_add(fill_escrow_result.added_amount).unwrap();
187 |                 user_escrow.yt_amount = user_escrow.yt_amount
188 |                     .checked_add(fill_escrow_result.added_amount).expect("YT overflow");
189 |             }
190 |         }
191 |     }
192 | }
```

The function contains an issue: it returns early when **staked_yt_amount** is zero. This prevents the user's **last_seen_sy_index** from being properly initialized at the moment they actually receive **YT** tokens.

```
/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
036 | /// Stage YT interest for user YT held in orderbook escrow's YT position
037 | /// This should be called whenever the user's YT position changes or before distributions
038 | /// current_sy_rate always needs to be fresh, fetched from the standard program
039 | pub fn stage_yt_interest(&mut self, current_sy_rate: Number) -> u64 {
040 |     if self.staked_yt_amount == 0 {
041 |         return 0;
042 |     }
043 |     let last_seen_sy_index = self.last_seen_sy_index.to_f64().unwrap();
044 |     if current_sy_rate <= self.last_seen_sy_index {
045 |         return 0;
046 |     }
047 |     // Calculate interest earned: YT_amount * (1/old_rate - 1/new_rate)
048 |     // This represents SY interest earned
049 |     let rate_delta = 1.0 / last_seen_sy_index - 1.0 / current_sy_rate.to_f64().unwrap();
050 |     let interest_earned = rate_delta * self.staked_yt_amount as f64;
051 |     let interest_earned_u64 = interest_earned.floor() as u64;
052 |     // Add to staged interest
```

```

057 |     self.staged_sy = self.staged_sy.checked_add(interest_earned_u64).unwrap();
059 |     // Update the user's last SY rate to current
060 |     self.last_seen_sy_index = current_sy_rate;
062 |     interest_earned_u64
063 | }

```

As a result, the interest index is set incorrectly, which causes inflated interest calculations and allows users to drain **SY** interest from the protocol.

Example:

- **At time T1:** the user creates a non-virtual **BuyYT** offer. A **UserEscrow** is created, and **last_seen_sy_index** is set to **current_sy_exchange_rate_1**.
- **At time T2:** the offer is filled and the **UserEscrow** receives **YT**. Because **staked_yt_amount** is still zero during the call, the function exits early, and **last_seen_sy_index** is not updated to **current_sy_exchange_rate_2**.
- **At time T3:** the exchange rate increases to **current_sy_exchange_rate_3**. When the user calls **collect_interest**, the protocol computes interest using the difference between **current_sy_exchange_rate_3** and **current_sy_exchange_rate_1**. The correct calculation should use the difference between **current_sy_exchange_rate_3** and **current_sy_exchange_rate_2**.

This leads to interest inflation and allows users to claim more **SY** than they are entitled to.

It is recommended to remove the **staked_yt_amount == 0** early return so that **last_seen_sy_index** is always updated when **YT** is first credited to the user.

This issue also exists in the **OrderbookFinancials.stage_yt_interest** function.

Resolution

This issue has been fixed by [aa77fa0](#).

EXPONENT_ORDERBOOK

[H-03] Missing prev_offer_idx reset causes cross-price linkage

Identified in commit [8c8b2db](#).

In the `RawOrderbook::find_counter_offer` function, `prev_offer_idx` is computed once at the beginning of the function and then reused across iterations that may advance to a different price node during the update at line 920. Furthermore, when the current price node is exhausted or empty (`idx == SENTINEL`), the code continues the outer loop without resetting `prev_offer_idx`, proceeding directly to the continue statement at Line 884.

When a valid offer is found in a subsequent new price node, the function returns `Some(FindCounterOfferOutputParams)` with `offer_idx` from the new node, but `prev_offer_idx` that may still belong to the previous node's linked list.

Later, during cleanup after fills, `remove_offer(prev_offer_idx, offer_idx, ...)` is invoked, which performs a raw relink `offers[prev].NEXT = offers[curr].NEXT` without validating that `prev` and `curr` belong to the same price node. This causes cross-price linked list corruption.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
839 | pub fn find_counter_offer<'a, I>(<
850 |     let mut prev_offer_idx = if filled_offers.next_offer_idx == SENTINEL {
851 |         SENTINEL
852 |     } else {
853 |         filled_offers.current_offer_idx
854 |     };
884 |     if idx == SENTINEL {
885 |         continue;
886 |     }
900 |     if offer.expiry_at > now {
901 |         filled_offers.current_price_node = pn;
902 |         // set pt_price_in_asset for next calculations in current price
903 |         if filled_offers.current_price_key != price_apy_counter {
904 |             filled_offers.current_pt_price_asset =
905 |                 calculate_price_in_assets_by_apy_pt(price_apy_counter, sec_remaining)
906 |                     .unwrap();
907 |         }
908 |         filled_offers.current_price_key = price_apy_counter;
909 |         filled_offers.next_offer_idx = node.get_register(NEXT_OFFER_REGISTER_USIZE);
910 |         filled_offers.current_offer_idx = idx;
911 |         return Some(FindCounterOfferOutputParams {
912 |             offer: *offer,
913 |             offer_idx: idx,
914 |             offer_price_key: price_apy_counter,
915 |             prev_offer_idx: prev_offer_idx,
916 |         });
917 |     }
919 |     filled_offers.next_offer_idx = node.get_register(NEXT_OFFER_REGISTER_USIZE);
920 |     prev_offer_idx = idx;
```

```

921 |         // move to the next offer in the linked list
922 |         idx = node.get_register(NEXT_OFFER_REGISTER_USIZE); // next_offer

```

For example (BuyYt traversals from low to high), with prices `P1`, `P2`, and `P3`:

- `P1: A1(expired) -> A2(expired) -> A3(expired) -> SENTINEL`
- `P2: SENTINEL`
- `P3: C1(valid) -> C2 -> SENTINEL`

`find_counter_offer` skips `A1`, `A2`, and `A3`. It ultimately leaves `prev_offer_idx = A3`, as the while loop progresses together with the assignment at Line 920.

It then observes that `P2` is empty (`idx == SENTINEL`) and proceeds without resetting `prev_offer_idx` at Lines 844-846. Upon entering `P3`, it immediately returns (`offer_idx = C1, prev_offer_idx = A3`).

Subsequently, the `fill_offer` function subtracts the matched amount from `offer.amount`. When the remaining amount falls below `configuration_options.threshold_amount`, `remove_offer` is invoked:

```

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
778 | if self
779 |     .offers
780 |     .get(counter_offer.offer_idx)
781 |     .get_value()
782 |     .is_empty(self.configuration_options.threshold_amount)
783 | {
784 |     self.remove_offer(
785 |         counter_offer.prev_offer_idx,
786 |         counter_offer.offer_idx,
787 |         sy_exchange_rate,
788 |         false,
789 |     )
790 |     .unwrap();
791 | }

```

During removal, with `prev = A3` and `curr = C1`, the following operation occurs: `offers[A3].NEXT = offers[C1].NEXT(C2)`.

This misalignment may allow a traversal of `P1` starting from `A1` to reach `C2`, causing an offer that originally belongs to `P3` to be treated as part of `P1`, thereby violating price boundaries and resulting in mismatched order matching across price levels.

It is recommended to set `prev_offer_idx` when switching to a new price node and before skipping empty nodes, so that any first valid node in a new price returns with `prev = SENTINEL`.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
863 | // iterate over counter offers SELL_YT in the price node
864 | let (&price_apy_counter, &pn) = if filled_offers.next_offer_idx == SENTINEL {
++   prev_offer_idx = SENTINEL;
865 |     price_iter.next().unwrap()
866 | } else {
867 |     (
868 |         &filled_offers.current_price_key,
869 |         &filled_offers.current_price_node,
870 |     )
871 | };
```

Resolution

This issue has been fixed by [3d0b12c](#).

EXPONENT_ORDERBOOK

[H-04] User escrow slot exhaustion

Identified in commit [8c8b2db](#).

The `remove_offer` routine in `solana/programs/exponent_orderbook/src/state/orderbook.rs` releases funds back to the user but never frees the associated `UserEscrow` slot when the escrow becomes empty and the per-user offer counter reaches zero. `UserEscrow` entries are only reclaimed via `withdraw_funds/collect_interest`, which most users never call after cancelling.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/withdraw_funds.rs */
189 | if user_escrow.is_empty()
190 |     && orderbook_raw
191 |         .user_escrows
192 |         .get_register(user_escrow_idx, USER_ESCROW_OFFER_COUNTER_REGISTER)
193 |         == 0
194 | {
195 |     orderbook_raw.remove_user_escrow(user_escrow_idx);
196 | }

/* solana/programs/exponent_orderbook/src/instructions/orderbook/collect_interest.rs */
096 | if user_escrow.is_empty()
097 |     && orderbook_raw
098 |         .user_escrows
099 |         .get_register(user_escrow_idx, USER_ESCROW_OFFER_COUNTER_REGISTER)
100 |         == 0
101 | {
102 |     orderbook_raw.remove_user_escrow(user_escrow_idx);
103 | }
```

Because the allocator is capped at `MAX_USER_ESCROWS = 250`, an attacker can cycle many throwaway wallets that each post-and-cancel a dust order, permanently consuming the escrow slots and preventing new users from trading.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1011 | pub fn get_or_create_user_escrow_idx(
1012 |     &mut self,
1013 |     user: Pubkey,
1014 |     sy_exchange_rate: Number,
1015 | ) -> Result<u32> {
1027 |     let new_user_escrow = UserEscrow::new(user, sy_exchange_rate);
1028 |     let new_idx = self.user_escrows.add_node(new_user_escrow);
1030 |     Ok(new_idx)
1031 | }

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
070 | pub struct RawOrderbook {
131 |     pub user_escrows: NodeAllocator<UserEscrow, MAX_USER_ESCROWS, 2>,

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
036 | const MAX_USER_ESCROWS: usize = 250;
```

All subsequent calls to `get_or_create_user_escrow_idx` will panic once the `RawOrderbook::user_escrows` runs out of free indices, effectively bricking every `post_offer/market_offer` instruction and rendering the orderbook inoperable through denial-of-service. The attack requires negligible capital. A sample scenario is:

1. Alice funds 250 throwaway wallets; each wallet posts the minimum-size order and immediately removes it, reclaiming its capital but leaving behind a `UserEscrow` entry.
2. Honest user Bob tries to trade, `get_or_create_user_escrow_idx` cannot allocate a new entry (`NodeALocator` bump hits `MAX_USER_ESCROWS`), causing the transaction to abort and the program to become unusable.

It's recommended to free empty escrows as soon as the last offer is removed, mirroring the clean-up that `withdraw_funds` already performs.

Resolution

This issue has been fixed by [2d747a2](#).

EXPONENT_ORDERBOOK

[M-01] Incorrect maker taker fee fields

Identified in commit [8c8b2db](#).

The `fill_offer` function in `solana/programs/exponent_orderbook/src/state/orderbook.rs` writes swap settlement data to `FilledOffersEvent`.

However, it mistakenly assigns the taker fees into the `maker_fee_amount` field and the maker fees into `taker_fee_amount`.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
929 | pub fn fill_offer(
930 |     &mut self,
931 |     taker_user_escrow_idx: u32,
932 |     offer_index: u32,
933 |     filled_amount: u64,
934 |     amount_sended: u64,
935 |     sy_exchange_rate: f64,
936 |     sy_exchange_rate_number: Number,
937 |     incoming_virtual_offer: bool,
938 |     fee_rates: &FeeRates,
939 | ) -> Result<StripAndMergeAmount> {
953 |     // fill taker escrow
954 |     let taker_fill_escrow_result = self.get_amounts_on_filling_escrow(
955 |         amount_sended,
956 |         filled_amount,
957 |         sy_exchange_rate,
958 |         offer_type.counter_type(),
959 |         incoming_virtual_offer,
960 |         true,
961 |         &fee_rates,
962 |     );
972 |     // fill seller escrow
973 |     let seller_fill_escrow_result = self.get_amounts_on_filling_escrow(
974 |         filled_amount,
975 |         amount_sended,
976 |         sy_exchange_rate,
977 |         offer_type,
978 |         virtual_offer,
979 |         false,
980 |         &fee_rates,
981 |     );
982 |
983 |     strip_and_merge_amounts.maker_fee_amount = taker_fill_escrow_result.fee_amount;
984 |     strip_and_merge_amounts.taker_fee_amount = seller_fill_escrow_result.fee_amount;
```

As a result, every emitted event misrepresents who actually paid which fee, and any accounting or incentive logic that trusts these fields will compute the wrong payouts or refunds.

The mismatch causes every downstream consumer of `FilledOffersEvent` to charge or credit maker/taker

incentives to the wrong party. If the off-chain settlement or rewards engine blindly trusts the event log, an attacker can consistently receive taker rebates while being a maker (or vice versa), leading to measurable financial loss for the protocol.

It's recommended to change the current assignment so that each fee amount is stored in its matching field:

```
strip_and_merge_amounts.maker_fee_amount = taker_fill_escrow_result.fee_amount;  
strip_and_merge_amounts.taker_fee_amount = seller_fill_escrow_result.fee_amount;
```

should be replaced with

```
strip_and_merge_amounts.maker_fee_amount = seller_fill_escrow_result.fee_amount;  
strip_and_merge_amounts.taker_fee_amount = taker_fill_escrow_result.fee_amount;
```

Resolution

This issue has been fixed by [8e38289](#).

EXPONENT_ORDERBOOK

[M-02] Rounding error in SY to PT/YT strip calculation

Identified in commit [8c8b2db](#).

In the orderbook's virtual `SellYT` mode, the program strips the `SY` provided by both the user and the counter into `PT` and `YT`. The user receives the `PT` corresponding to the base value of their contributed `SY`, and the counterparty receives the `YT` corresponding to the base value of their contributed `SY`.

In the strip process, the "strip SY value" is equal to the `PT/YT` base amount, and the `SY` value is defined as `SY_token_amount * sy_exchange_rate`.

When the orderbook program computes the actual `SY` amount to strip, it converts the target `SY` value into an `SY` token amount via the `calculate_sy_amount_from_base` function. In this function, `taken_amount` represents the `SY` value (in base units), and `sy_exchange_rate` is the `SY` token's exchange rate.

```
/* solana/programs/exponent_orderbook/src/state/offer.rs */
138 |     let taken_amount_sy = calculate_sy_amount_from_base(taken_amount, sy_exchange_rate);
139 |     offer.amount = offer
140 |         .amount
141 |         .checked_add(filled_amount)
142 |         .unwrap()
143 |         .checked_sub(taken_amount_sy) // cause taken YT to SY after merge
144 |         .unwrap();
145 |
146 |     StripAndMergeAmount {
147 |         strip_sy_amount: taken_amount_sy,
148 |         merged_base_amount: 0,
149 |         taker_fee_amount: 0,
150 |         maker_fee_amount: 0,
151 |     }
152 | }
```

The target `SY` value is treated as the increase in `PT/YT` balance that the user or counterparty should receive in their `user_escrow`. After the `SY` is stripped, `orderbook.financials` is updated with the global `PT/YT` delta using the result returned from the `strip_and_merge_result` call (the actual `strip` instruction return value).

```
/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
180 | self.financials.yt_fee_balance = self.financials.yt_fee_balance
    |     .checked_add(fill_escrow_result.fee_amount).unwrap();
181 | user_escrow.stage_yt_interest(sy_exchange_rate);
182 | user_escrow.staked_yt_amount = user_escrow.staked_yt_amount
    |     .checked_add(fill_escrow_result.added_amount).unwrap();
183 | user_escrow.yt_amount = user_escrow.yt_amount.checked_add(fill_escrow_result.added_amount)
    |     .expect("YT overflow");
186 | OfferType::SellYt => {
```



```

187 |     if virtual_offer {
188 |         self.financials.pt_fee_balance = self.financials.pt_fee_balance
                                           .checked_add(fill_escrow_result.fee_amount).unwrap();
189 |         user_escrow.pt_amount = user_escrow.pt_amount.checked_add(fill_escrow_result.added_amount
                                           .expect("PT overflow"));

/* solana/programs/exponent_orderbook/src/instructions/orderbook/post_offer.rs */
320 | // Do strip and merge
321 | let strip_and_merge_result = do_strip_and_merge(
322 |     &strip_and_merge_amount,
323 |     &ctx.accounts
324 |         .cpi_account_orderbook
325 |         .exponent_core_cpi_accounts,
326 |     &ctx.accounts.cpi_account_orderbook.sy_cpi_accounts,
327 |     ctx.remaining_accounts,
328 |     &ctx.accounts.to_account_infos(),
329 |     signer_seeds,
330 |     ctx.accounts.address_lookup_table.to_account_info(),
331 |     *ctx.accounts.exponent_core.key,
332 |     sy_exchange_rate,
333 | )?;
334 |
335 | {
336 |     let mut data = ctx.accounts.orderbook.data.borrow_mut();
337 |     let orderbook_raw: &mut RawOrderbook = bytemuck::from_bytes_mut(&mut data[8..]);
338 |     orderbook_raw
339 |         .financials
340 |         .update_on_strip_and_merge(&strip_and_merge_result);

```

However, `calculate_sy_amount_from_base` rounds the SY token amount to be stripped downwards. This means the actual stripped SY token amount is smaller than the ideal value, causing the resulting PT/YT amount to be lower than expected. As a result, the protocol's internal accounting becomes inconsistent and may introduce a potential underflow risk.

```

/* solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs */
167 | pub fn calculate_sy_amount_from_base(amount: u64, sy_exchange_rate: f64) -> u64 {
168 |     (amount as f64 / sy_exchange_rate).floor() as u64
169 | }

```

Example (fees are ignored for simplicity):

- The `taken_amount` (SY value) is 100, and `sy_exchange_rate` is 1.9.
- The user or counter is expected to receive PT/YT with a base value of 100.
- The actual stripped SY token amount is $\text{floor}(100 / 1.9) = 52$.
- In the core program, the strip result is: $\text{PT/YT amount} = \text{SY_token_amount} * \text{sy_exchange_rate} = 52 * 1.9 = 98$.

So the protocol actually receives only 98 units of PT/YT, and `orderbook.financials` records an increase of 98, but the `user_escrow` records an increase of 100.

It is recommended to update the `taken_amount_sy` calculation to use ceiling rounding.

```
/* solana/programs/exponent_orderbook/src/state/offer.rs */
138 |     let taken_amount_sy = calculate_sy_amount_from_base(taken_amount, sy_exchange_rate);
139 |     offer.amount = offer
140 |         .amount
141 |         .checked_add(filled_amount)
142 |         .unwrap()
143 |         .checked_sub(taken_amount_sy) // cause taken YT to SY after merge
144 |         .unwrap();
146 |     StripAndMergeAmount {
147 |         strip_sy_amount: taken_amount_sy,
148 |         merged_base_amount: 0,
149 |         taker_fee_amount: 0,
150 |         maker_fee_amount: 0,
151 |     }
152 | }
```

Resolution

This issue has been fixed by `cd2e58f`.

EXPONENT_ORDERBOOK

[M-03] Crossed book risk caused by post_offer match limit

Identified in commit [8c8b2db](#).

In `post_offer`, there is a hard cap `MAX_FILLED_OFFERS` on how many counter orders can be matched. When the number of filled counter offers reaches this limit, the instruction does not fail. Instead, it stops matching and turns the remaining tokens into a new offer that is inserted into the orderbook.

In this situation, better counter offers are still present in the book (for an incoming `BuyYt`, lower-priced `SellYt` offers may still exist, and vice versa). Because those better offers were not fully consumed, the newly inserted residual offer can end up inside the existing counter side's price range, causing `BuyYt` and `SellYt` ranges to overlap.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
416 | pub fn post_offer(
425 | ) -> Result<PostOfferOutput> {
444 | {
445 |     let price_iter = &mut self.get_price_iterator(offer_type).peekable();
446 |     while remaining_amount > self.configuration_options.threshold_amount
447 |         && MAX_FILLED_OFFERS > filled_counter
448 |     {
450 |         let find_counter_offer_output_params_opt = self.find_counter_offer(
456 |         );
459 |         if let Some(find_counter_offer_output_params) = find_counter_offer_output_params_opt
460 |         {
461 |             filled_counter += 1;
536 |         } else {
537 |             break;
538 |         }
539 |     }
540 | }
605 | // If anything left, insert a new residual offer
606 | let offer_idx = if remaining_amount > self.configuration_options.threshold_amount {
607 |     let new_offer_idx = self
608 |         .add_offer(
616 |         )
617 |         .unwrap();
624 | };
634 | }
```

The main impact is negative for the order placer. The new residual offer can no longer consume counter offers that are better than the submitted `price_apy`, even if those offers already exist in the orderbook.

In addition, other users can consume the remaining better counter offers first and then consume the crossed offer, capturing risk-free spread by extracting value from the crossed price behavior.

This behavior can also be exploited intentionally. For ease of discussion, assume the victim's order is a `B`

uyYt.

Because `post_offer` provides no slippage protection, users cannot enforce a minimum expected output. When a normal user calls `post_offer` to consume `SellYt` offers priced better than `price_apy` and expects the leftover funds to become a new offer, an attacker can place many small `SellYt` offers at the current best `SellYt` price level (or slightly lower, denoted as `best_price_apy`). This can cause `post_offer` to hit the `MAX_FILLED_OFFERS` limit while consuming only a small total amount of the best liquidity, leaving a large remainder that becomes a higher-priced `BuyYt` resting offer. The attacker can then consume the remaining lower-priced `SellYt` offers and finally trade against the crossed `BuyYt` offer created by `post_offer`.

This attack requires a sandwich-style setup. However, it can be executed with minimal capital, without the need to sweep the order book with substantial funds. And even if it fails, for example, the attacker's small offers are posted after the victim's `post_offer`, the loss will be small, because the attacker's offers will trade against the best available `BuyYt` in the book. Since the victim's `post_offer` likely already consumed offers in the `best_price_apy` to `price_apy` range, and assuming no other orders were inserted in between, the attacker's fill price would likely be close to `price_apy`, which is generally a fair price. As a result, the attacker's downside may be limited. When the victim's `post_offer` size is large and the price band is wide, an attacker may have strong incentives to attempt this strategy, effectively leveraging small capital for outsized gains.

It is recommended to return the remaining funds instead of creating a new offer when the `MAX_FILLED_OFFERS` limit is hit, or at least to add an explicit option to control this behaviour so users understand the risk and can opt out. Another solution is to add a slippage parameter to the `post_offer`, similar to `min_amount_out` in the `market_offer`. This ensures that the amount executed immediately meets user expectations, preventing more funds than intended from being tied up in resting orders.

Resolution

This issue has been fixed by [ec0183a](#).

EXPONENT_ORDERBOOK

[M-04] Desynchronized SY exchange rates cause yield loss or DoS

Identified in commit [8c8b2db](#).

The `orderbook` program currently calculates YT interest by directly querying the `sy_program` for the real-time `sy_exchange_rate`. It maintains its own internal interest state via `UserEscrow::stage_yt_interest` and `OrderbookFinancials::stage_yt_interest`.

Using the `post_offer` instruction as an example, we can see that the `orderbook` independently maintains `last_sy_exchange_rate` and uses it within `UserEscrow` and `OrderbookFinancials`.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/post_offer.rs */
245 | pub fn handler<'info>(<
253 | ) -> Result<PostOfferEvent> {
254 |     // get exchange rate for SY
255 |     let sy_exchange_rate_from_sy = do_get_sy_state(
256 |         &ctx.accounts.address_lookup_table,
257 |         &ctx.accounts.cpi_account_orderbook.sy_cpi_accounts,
258 |         ctx.remaining_accounts,
259 |         ctx.accounts.sy_program.key(),
260 |     )
261 |     .map(|x| x.exchange_rate)
262 |     .expect("failed to get sy state");
267 |     let orderbook_raw: &mut RawOrderbook = bytemuck::from_bytes_mut(&mut data[8..]);
268 |     let sy_exchange_rate = orderbook_raw.validate_sy_exchange_rate(sy_exchange_rate_from_sy)?;
272 |     orderbook_raw.financials.stage_yt_interest(sy_exchange_rate);
365 | }

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1255 | pub fn validate_sy_exchange_rate(&mut self, sy_exchange_rate: Number) -> Result<Number> {
1256 |     let sec_remaining = self.financials.get_sec_remaining()?;
1257 |     // @audit: lastest rate before expired
1258 |     if sec_remaining == 0 {
1259 |         return Ok(self.last_sy_exchange_rate);
1260 |     }
1267 |     self.last_sy_exchange_rate = sy_exchange_rate;
1268 |
1269 |     Ok(sy_exchange_rate)
1270 | }

/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
038 | pub fn stage_yt_interest(&mut self, current_sy_rate: Number) -> u64 {
055 |     // Add to staged interest
056 |     self.staged_sy = self.staged_sy.checked_add(interest_earned_u64).unwrap();
057 |
058 |     // @audit: current_sy_rate is from validate_sy_exchange_rate
059 |     // Update the user's last SY rate to current
060 |     self.last_seen_sy_index = current_sy_rate;
061 | }

/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
136 | pub fn stage_yt_interest(&mut self, current_sy_rate: Number) -> u64 {
```

```

153 | // Add to staged interest
154 | self.staged_sy = self.staged_sy.checked_add(interest_earned_u64).unwrap();
155 |
    | // @audit: current_sy_rate is from validate_sy_exchange_rate
156 | // Update the user's last SY rate to current
157 | self.last_seen_sy_index = current_sy_rate;
160 | }

```

However, since `stage_yt_interest` on the `orderbook` side does not call `stage_yt_yield` in the `core` program, there is no guarantee that the exchange rates in both programs are synchronized.

In normal scenarios, this does not cause issues because when the user and the admin want to claim interest, they will call `collect_interest` or `collect_admin_fee` on the `orderbook` side, which will eventually trigger `stage_yt_yield` in the `core` program. This action updates both programs to the same `sy_exchange` value at that specific moment.

The situation becomes critical due to the expiration logic. Consider a scenario where, just before PT/YT expiry, the `sy_exchange` in the `core` program is last updated at timestamp A, while the `orderbook` updates at timestamp B. These two timestamps (and thus rates) can differ because the `core` program can be updated independently, and the `orderbook` does not consistently trigger a `core` update.

Once expired, subsequent calls to `collect_interest` or `collect_admin_fee` will no longer update the `sy_exchange` rate. This permanently freezes the mismatch between the accumulated interest in the `orderbook` and the `core` program. This mismatch depends on the timing of A and B:

- If the `orderbook` rate is lower, users and the admin suffer yield loss.
- If the `orderbook` rate is higher, the claim transaction may revert (causing a DoS) because the `core` vault does not have enough funds to cover the inflated interest recorded by the `orderbook`.

We recommend stopping the direct query of `sy_program`. Instead, the system should invoke `stage_yt_yield` to update the `core` program's rate during interactions. Furthermore, after expiration, the `orderbook` should retrieve and use the `final_sy_exchange_rate` stored in the `core` program's vault to ensure consistency.

Resolution

This issue has been fixed by `70077e6` and `58bb45b`.

EXPONENT_ORDERBOOK

[M-05] Invalid escrow lookup breaks offer cancellation

Identified in commit [acbc61e](#).

In `remove_offer.rs`, the `remove_offer` derives the maker's `user_escrow_idx` after the offer is removed from the raw orderbook. However, it uses the wrong allocator and an invalid register ID.

In particular, the `RawOrderbook::remove_offer` already retrieves the correct escrow index from the offer allocator before deletion:

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1309 | let user_escrow_idx = self.offers.get_register(offer_idx, USER_ESCROW_REGISTER);
```

However, the outer handler later attempts to rediscover that same value with:

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/remove_offer.rs */
254 | let (offer_type_flag, offer_amount, user_escrow_idx) = {
277 |     (
278 |         offer.offer_type_flag,
279 |         offer.amount,
280 |         orderbook_raw.user_escrows.get_register(offer_idx, USER_ESCROW_REGISTER),
281 |     )
282 | };
```

This is incorrect because `offer_idx` is an index into `offers`, not into `user_escrows`.

In addition, `USER_ESCROW_REGISTER` is `2`, while `user_escrows` is defined as `NodeAllocator<UserEscrow, MAX_USER_ESCROWS, 2>`, which means only registers `0` and `1` exist. Therefore, accessing register `2` is out of bounds.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
030 | const NEXT_OFFER_REGISTER: u32 = 1;
031 | const NEXT_OFFER_REGISTER_USIZE: usize = NEXT_OFFER_REGISTER as usize;
032 | pub const USER_ESCROW_REGISTER: u32 = 2;
033 | const OFFER_PRICE_REGISTER: u32 = 3;

120 | /// Offer slots (fixed capacity)
121 | /// Link to offers pointers in the RedBlackTree
122 | /// Every PriceNode has a two pointers to sell and buy linked lists with offers
123 | /// Each offer has a pointer to the next offer in the list
124 | pub offers: NodeAllocator<
125 |     Offer,
126 |     MAX_OFFERS, // capacity
127 |     4,
128 | >,
129 |
130 | /// All possible user escrows (fixed capacity)
```

```
131 | /// second register is a counter of offers for this user
132 | pub user_escrows: NodeAllocator<UserEscrow, MAX_USER_ESCROWS, 2>,
```

As a result, `remove_offer` deterministically reverts during cancellation instead of completing the maker's withdrawal flow. Makers cannot reliably cancel live offers through either `remove_offer` or `wrapper_remove_offer` until a taker fills the order, or the offer expires and is removed by an admin.

It is recommended to update the code as follows:

```
let user_escrow_idx = orderbook_raw.offers.get_register(offer_idx, USER_ESCROW_REGISTER);
let offer = orderbook_raw.remove_offer(previous_offer_idx, offer_idx, sy_exchange_rate, true).unwrap();
(
    offer.offer_type_flag,
    offer.amount,
    user_escrow_idx,
)
```

Resolution

This issue has been fixed by [1ebf675](#).

EXPONENT_ORDERBOOK

[L-01] Missing admin checks in orderbook initialization instructions

Identified in commit [8c8b2db](#).

The `orderbook_init` and `realloc_orderbook_account` instructions are supposed to be called by admins as they are in the `admin` folder, and they are marked as "High-trust functions" in comments.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/README.md */
011 | 3 Create the orderbook itself, which involves two sub-steps
012 |     3.1 Use the realloc_orderbook_account instruction to allocate the necessary memory for the orderbook
    ↪ account. Each invocation of this instruction increases the account size by 10240 bytes (Solana limit)
013 |     3.2 Initialize the orderbook itself by invoking orderbook_init
```

Right now, anyone can create an order book, which seems inconsistent with the intention.

Resolution

The `High-trust functions` comments were removed by commit [289bd44](#) so that these two instructions are designed to be permissionless.

EXPONENT_ORDERBOOK

[L-02] Unsafe f64 to integer casting risks

Identified in commit [8c8b2db](#).

During several protocol calculations, the code casts `f64` values into `u32` or `u64` using the `as` keyword, which is not type safe and may introduce silent precision loss.

1. f64 as u32

In `get_modified_price_decimals`, the function casts the `f64` value `rounded_price` into a `u32`.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1272 | pub fn get_modified_price_decimals(&self, price_apy: f64) -> u32 {
1273 |     let decimals = self.configuration_options.price_decimals as i32;
1274 |     let factor = 10_f64.powi(decimals);
1275 |     let rounded_price = (price_apy * factor).round() / factor;
1276 |
1277 |     (rounded_price * PRICE_BASE_POINTS as f64).round() as u32
1278 | }
```

Since the numeric range of `f64` is much larger than `u32`, this conversion can lead to unexpected truncation.

2. f64 as u64

In `stage_yt_interest`, the function casts the `f64` value `interest_earned` into a `u64`.

```
/* solana/programs/exponent_orderbook/src/state/user_escrow.rs */
136 | pub fn stage_yt_interest(&mut self, current_sy_rate: Number) -> u64 {
137 |     if self.yt_fee_balance == 0 {
138 |         return 0;
139 |     }
140 |
141 |     let last_seen_sy_index = self.last_seen_sy_index.to_f64().unwrap();
142 |
143 |     if current_sy_rate <= self.last_seen_sy_index {
144 |         return 0;
145 |     }
146 |
147 |     // Calculate interest earned: YT_amount * (1/old_rate - 1/new_rate)
148 |     // This represents SY interest earned
149 |     let rate_delta = 1.0 / last_seen_sy_index - 1.0 / current_sy_rate.to_f64().unwrap();
150 |     let interest_earned = rate_delta * self.yt_fee_balance as f64;
151 |     let interest_earned_u64 = interest_earned.floor() as u64;
```

However, `f64` cannot precisely represent integers larger than 2^{53} , so when `interest_earned` exceeds this threshold, precision loss occurs, resulting in an inaccurate `u64` conversion.

It is recommended to add proper bounds checks and type-safe conversions before casting, ensuring that

the intermediate `f64` values remain within safe integer ranges and avoiding silent precision loss.

Resolution

This issue has been fixed by `2921277`.

EXPONENT_ORDERBOOK

[L-03] Precision loss accumulation causes SY deficit

Identified in commit [8c8b2db](#).

In the orderbook's virtual `SellYT` mode, when the user acts as a taker and posts a `SellYT` offer to make offer, both the user's `SY` tokens and the counterparty's `SY` tokens are stripped into `PT` and `YT`.

The user's actual `SY` input is computed as the difference: `amount - post_offer_output.remaining_amount`. The counterparty's `SY` tokens are pre-deposited in the order book.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/market_offer.rs */
285 | // subtract that remaining from filling amount
286 |     let send_amount = amount
287 |         .checked_sub(post_offer_output.remaining_amount)
288 |         .unwrap();
```

During each match, `post_offer_output.remaining_amount` is updated using: `remaining_amount = remaining_amount + taken_from_counter_offer_amount - (filled_from_amount / sy_exchange_rate)`

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs */
145 | if virtual_offer {
146 |     if offer_type == OfferType::SellYt {
147 |         remaining_amount
148 |         .checked_add(taken_from_counter_offer_amount)
149 |         .unwrap()
150 |         .checked_sub(calculate_sy_amount_from_base(
151 |             filled_from_amount,
152 |             sy_exchange_rate,
153 |         ))
154 |         .unwrap()
155 |     }

/* solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs */
167 | pub fn calculate_sy_amount_from_base(amount: u64, sy_exchange_rate: f64) -> u64 {
168 |     (amount as f64 / sy_exchange_rate).floor() as u64
169 | }
```

- `taken_from_counter_offer_amount` is the `SY` token amount supplied by the counterparty
- `filled_from_amount` is the `SY` base value that will be stripped
- `sy_exchange_rate` is the `SY` token's exchange rate
- `(filled_from_amount / sy_exchange_rate)` converts the base `SY` value into the actual `SY` token amount that must be stripped

As a result, the user's actual `SY` input equals the sum over all fills of: `(filled_from_amount / sy_exchange_rate) - taken_from_counter_offer_amount`.

Meanwhile, after each fill, `filled_from_amount` is accumulated into `total_counter_filled_amount`. The final SY token amount to be stripped is computed once at the end as: `total_counter_filled_amount / sy_exchange_rate_float`

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
715 | total_counter_filled_amount =
716 |     total_counter_filled_amount.checked_add(filled).unwrap();

819 |     strip_and_merge_amount.strip_sy_amount = strip_and_merge_amount
820 |         .strip_sy_amount
821 |         .checked_add(calculate_sy_amount_from_base(
822 |             total_counter_filled_amount,
823 |             sy_exchange_rate_float,
824 |         ))
825 |         .unwrap();
826 | }
```

Because a single maker offer may fill multiple counter offers, precision loss accumulates. Each fill performs `filled_from_amount / sy_exchange_rate`, introducing rounding loss multiple times. However, `total_counter_filled_amount / sy_exchange_rate_float` is performed only once at the end, causing a mismatch.

As a result, the user's actual SY input + counterparty SY input can be less than the final SY amount consumed by strip, leading to potential protocol insolvency.

Example: A user creates a maker offer with `amount = 250` and two fills occur.

- Assume `sy_exchange_rate = 1.9`.
- Order 1
 - `filled_from_amount = 100`
 - `taken_from_counter_offer_amount = 20`
 - `total_counter_filled_amount = 100`
 - `remaining_amount = 250 + 20 - 100 / 1.9 = 218`
- Order 2
 - `filled_from_amount = 100`
 - `taken_from_counter_offer_amount = 20`
 - `total_counter_filled_amount = 200`
 - `remaining_amount = 218 + 20 - 100 / 1.9 = 186`
- User actual SY sent: `250 - 186 = 64`
- Counterparty SY provided: `20 + 20 = 40`

- Protocol expects to consume: $66 + 40 = 104$
- Actual strip SY amount: $200 / 1.9 = 105$

The user's actual SY input is less than the strip operation's required SY amount.

It is recommended to update all three instances of the `calculate_sy_amount_from_base` function to use ceiling rounding:

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/util.rs */
148 | remaining_amount
149 |     .checked_add(taken_from_counter_offer_amount)
150 |     .unwrap()
151 |     .checked_sub(calculate_sy_amount_from_base(
152 |         filled_from_amount,
153 |         sy_exchange_rate,
154 |     ))
155 |     .unwrap()

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
595 |     strip_and_merge_amount.strip_sy_amount = strip_and_merge_amount
596 |         .strip_sy_amount
597 |         .checked_add(calculate_sy_amount_from_base(
598 |             total_counter_filled_amount,
599 |             sy_exchange_rate_float,
600 |         ))
601 |         .unwrap();
602 | }

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
819 |     strip_and_merge_amount.strip_sy_amount = strip_and_merge_amount
820 |         .strip_sy_amount
821 |         .checked_add(calculate_sy_amount_from_base(
822 |             total_counter_filled_amount,
823 |             sy_exchange_rate_float,
824 |         ))
825 |         .unwrap();
826 | }
```

Resolution

This issue has been fixed by [c8cbc4a](#).

EXPONENT_ORDERBOOK

[L-04] SY rounding misguides merge/strip

Identified in commit [8c8b2db](#).

During the offer fill process, if both the taker and the seller provide virtual offers, then both `strip_and_merge_amount.merged_base_amount` and `strip_and_merge_amount.strip_sy_amount` will be non-zero.

Inside `do_strip_and_merge`, when both values are non-zero, the function converts `merged_base_amount` into its corresponding SY amount and compares it with `strip_sy_amount` to determine whether to execute a `merge` or a `strip`. If the converted SY amount equals `strip_sy_amount`, the function performs neither merge nor strip.

```

/* solana/programs/exponent_orderbook/src/instructions/orderbook/market_offer.rs */
399 | // recalculate merged amount in sy to check diff
400 |     let merged_sy_amount = calculate_sy_amount_from_base_number(
401 |         strip_and_merge_amount.merged_base_amount,
402 |         sy_exchange_rate,
403 |     );
404 |
405 |     let strip_or_merge_result = if merged_sy_amount > strip_and_merge_amount.strip_sy_amount {
406 |         let strip_base_amount = calculate_base_amount_from_sy_number(
407 |             strip_and_merge_amount.strip_sy_amount,
408 |             sy_exchange_rate,
409 |         );
410 |         let diff_merge_amount = strip_and_merge_amount
411 |             .merged_base_amount
412 |             .checked_sub(strip_base_amount)
413 |             .unwrap();
414 |
415 |         // Withdraw YT from the vault for merging with PT
416 |         do_cpi_withdraw_yt(
417 |             &address_lookup_table,
418 |             exponent_core_cpi_accounts,
419 |             cpi_sy_accounts,
420 |             remaining_accounts,
421 |             regular_accounts,
422 |             exponent_core_program,
423 |             diff_merge_amount,
424 |             &[&signer_seeds],
425 |         )
426 |         .unwrap();
427 |
428 |         let merge_result = do_cpi_merge_sy(
429 |             diff_merge_amount,
430 |             &address_lookup_table,
431 |             exponent_core_cpi_accounts,
432 |             regular_accounts,
433 |             remaining_accounts,
434 |             exponent_core_program,
435 |             &[&signer_seeds],
436 |         )

```

```

437 |         .unwrap();
438 |
439 |         StripOrMergeResult {
440 |             amount: diff_merge_amount,
441 |             out_amount: merge_result.amount_sy_out,
442 |             direction: TransformSyDirection::Merge,
443 |         }
444 |     } else {
445 |         if merged_sy_amount != strip_and_merge_amount.strip_sy_amount {
446 |             let diff_strip_amount = strip_and_merge_amount
447 |                 .strip_sy_amount
448 |                 .checked_sub(merged_sy_amount)
449 |                 .unwrap();
450 |
451 |             let strip_result = do_cpi_strip_sy(
452 |                 diff_strip_amount,
453 |                 &address_lookup_table,
454 |                 exponent_core_cpi_accounts,
455 |                 regular_accounts,
456 |                 remaining_accounts,
457 |                 exponent_core_program,
458 |                 &[&signer_seeds],
459 |             )?;
460 |
461 |             // Deposit YT to the vault after stripping SY
462 |             do_cpi_deposit_yt(
463 |                 &address_lookup_table,
464 |                 exponent_core_cpi_accounts,
465 |                 cpi_sy_accounts,
466 |                 remaining_accounts,
467 |                 regular_accounts,
468 |                 strip_result.amount_py_out,
469 |                 exponent_core_program,
470 |                 &[&signer_seeds],
471 |             )
472 |             .unwrap();
473 |             StripOrMergeResult {
474 |                 amount: diff_strip_amount,
475 |                 out_amount: strip_result.amount_py_out,
476 |                 direction: TransformSyDirection::Strip,
477 |             }
478 |         } else {
479 |             StripOrMergeResult {
480 |                 amount: 0,
481 |                 out_amount: 0,
482 |                 direction: TransformSyDirection::Strip,
483 |             }
484 |         }
485 |     };

```

However, because the comparison is performed in the **SY** amount space, which involves integer rounding when converting from base to **SY**, the **merge/strip** decision may be made at a rounded boundary.

In such cases, two **SY** amounts can appear equal even though their underlying base values still differ by one minimal unit, leading to a **merge/strip** decision that diverges from the intended protocol behavior.

Assume:

```
sy_exchange_rate = 1.9
merged_base_amount = 29
strip_sy_amount = 15

// current logic
merged_sy_amount = merged_base_amount / sy_exchange_rate = floor(29 / 1.9) = 15
// SY side sees: merged_sy_amount == strip_sy_amount -> treat as equal

// but when we convert strip_sy_amount back to base with integer math
strip_base_amount = floor(strip_sy_amount * sy_exchange_rate) = floor(15 * 1.9) = 28

// so on the base side we still have 1 unit difference: 29 vs 28
```

It is recommended to compare the base value reconstructed from `strip_sy_amount` against `merged_base_amount`.

This does not remove rounding entirely, but it ensures the `merge/strip` decision is based on the same rounding convention as vault accounting in base units, and fixes the case where `merged_sy_amount == strip_sy_amount` but their base values still differ by one minimal unit.

Resolution

This issue has been fixed by [da55473](#).

EXPONENT_ORDERBOOK

[L-05] Core program Strip/Merge limit blocks virtual-virtual matching

Identified in commit [8c8b2db](#).

In the `orderbook` program's `post_offer` and `market_offer` instructions, if the matched orders include virtual offers, the program calls the `core` program's `merge` or `strip` instruction as needed based on the final difference between merge and strip.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/market_offer.rs */
388 | pub fn do_strip_and_merge<'info>(<
398 | ) -> Result<StripOrMergeResult> {
405 |     let strip_or_merge_result = if merged_sy_amount > strip_and_merge_amount.strip_sy_amount {
410 |         let diff_merge_amount = strip_and_merge_amount
411 |             .merged_base_amount
412 |             .checked_sub(strip_base_amount)
413 |             .unwrap();
428 |         let merge_result = do_cpi_merge_sy(
429 |             diff_merge_amount,
430 |             &address_lookup_table,
431 |             exponent_core_cpi_accounts,
432 |             regular_accounts,
433 |             remaining_accounts,
434 |             exponent_core_program,
435 |             &[&signer_seeds],
436 |         )
437 |         .unwrap();
444 |     } else {
445 |         if merged_sy_amount != strip_and_merge_amount.strip_sy_amount {
446 |             let diff_strip_amount = strip_and_merge_amount
447 |                 .strip_sy_amount
448 |                 .checked_sub(merged_sy_amount)
449 |                 .unwrap();
450 |
451 |             let strip_result = do_cpi_strip_sy(
452 |                 diff_strip_amount,
453 |                 &address_lookup_table,
454 |                 exponent_core_cpi_accounts,
455 |                 regular_accounts,
456 |                 remaining_accounts,
457 |                 exponent_core_program,
458 |                 &[&signer_seeds],
459 |             )?;
485 |     };
488 | }
```

But in the `core` program's `vault`, the admin can set a minimum amount per `strip` or `merge`. When the difference needed after matching is very small, it can be below this limit, the CPI call fails, and the whole transaction fails, so the user request cannot be completed.

```

/* solana/programs/exponent_core/src/instructions/vault/merge.rs */
138 | require!(
139 |     self.vault.is_min_op_size_merge(amount_py),
140 |     ExponentCoreError::OperationAmountTooSmall
141 | );

/* solana/programs/exponent_core/src/instructions/vault/strip.rs */
140 | require!(
141 |     self.vault.is_min_op_size_strip(amount),
142 |     ExponentCoreError::OperationAmountTooSmall
143 | );

```

Because the program has a lot of rounding, this is very likely to happen when both the incoming offer and the counter offer are virtual.

In this case, in theory no `merge` or `strip` should happen, because a successful match is equal to a direct SY to PT trade. It does not need YT and does not require splitting or merging SY.

But to simplify matching between any virtual and non-virtual orders, the program converts everything to traded YT before matching. Rounding down in this step makes two virtual orders fail to match exactly. Also, even when both are virtual, they still contribute separately to `strip_and_merge_amount.strip_sy_amount` and `strip_and_merge_amount.merged_base_amount` instead of canceling each other, which will introduce more rounding error. When it reaches `do_strip_and_merge`, the merge and strip difference might not be zero but a tiny dust amount, which can easily hit the minimum `merge` or `strip` limit in the `core` program, causing a rollback and stopping some normal offers from executing.

Below is an example. The incoming offer is `OfferType::BuyYt` with `virtual_offer = true`. For simplicity, assume there is one counter offer that matches the price and has enough amount. We also provide `pt_price_asset` directly, and do not compute it from `price_apy` and `sec_remaining`, which is more direct.

```

#[cfg(test)]
mod tests {
    use super::*;
    use crate::instructions::{calculate_base_amount_from_sy_number, calculate_sy_amount_from_base_number};
    use crate::instructions::util::get_available_amount;
    use crate::OfferType;

    #[test]
    fn virtual_virtual_single_fill_rounding_diff() {
        let amount = 33u64;
        let offer_type = OfferType::BuyYt;
        let virtual_offer = true;
        let sy_exchange_rate = Number::from_ratio(11, 10); // 1.1

        let sy_exchange_rate_float = sy_exchange_rate.to_f64().unwrap();

        // we provide pt_price_asset, so price_apy and sec_remaining is not used
        let price_apy = 0u32;
        let sec_remaining = 0u32;
    }

```

```

let pt_price_asset = 0.91_f64;

let mut remaining_amount = amount;

let counter_sy_amount = 1000u64;

let quoted_amount = compute_quote_amount(
    offer_type,
    remaining_amount,
    price_apy,
    sy_exchange_rate_float,
    sec_remaining,
    virtual_offer,
    Some(pt_price_asset),
)
.unwrap();

let available_amount = get_available_amount(
    price_apy,
    sec_remaining,
    sy_exchange_rate_float,
    offer_type.counter_type(),
    counter_sy_amount,
    true,
    Some(pt_price_asset),
);

let taken_amount = if available_amount > quoted_amount {
    quoted_amount
} else {
    available_amount
};

let filled = compute_quote_amount_no_virtual(
    offer_type.counter_type(),
    taken_amount,
    price_apy,
    sy_exchange_rate_float,
    sec_remaining,
    Some(pt_price_asset),
)
.unwrap();

remaining_amount = calculate_remaining_amount_on_counter_offer_filling(
    virtual_offer,
    offer_type,
    remaining_amount,
    filled,
    taken_amount,
    sy_exchange_rate_float,
);

assert_eq!(taken_amount, 33);
assert_eq!(filled, 2);
assert_eq!(remaining_amount, 0);

let taken_amount_sy = calculate_sy_amount_from_base(taken_amount, sy_exchange_rate_float);

let mut strip_and_merge_amount = StripAndMergeAmount {
    strip_sy_amount: taken_amount_sy,

```

```

        merged_base_amount: 0,
        taker_fee_amount: 0,
        maker_fee_amount: 0,
    };

    let diff_with_amount = amount.checked_sub(remaining_amount).unwrap();
    strip_and_merge_amount.merged_base_amount = strip_and_merge_amount
        .merged_base_amount
        .checked_add(diff_with_amount)
        .unwrap();

    let merged_sy_amount = calculate_sy_amount_from_base_number(
        strip_and_merge_amount.merged_base_amount,
        sy_exchange_rate,
    );
    assert!(merged_sy_amount > strip_and_merge_amount.strip_sy_amount);

    let strip_base_amount = calculate_base_amount_from_sy_number(
        strip_and_merge_amount.strip_sy_amount,
        sy_exchange_rate,
    );

    let diff_merge_amount = strip_and_merge_amount
        .merged_base_amount
        .checked_sub(strip_base_amount)
        .unwrap();

    assert!(diff_merge_amount > 0);
    println!("strip_and_merge_amount:");
    println!("    .strip_sy_amount: {}", strip_and_merge_amount.strip_sy_amount);
    println!("    .merged_base_amount: {}", strip_and_merge_amount.merged_base_amount);
    println!("    strip_base_amount: {}", strip_base_amount);
    println!("diff_merge_amount: {}", diff_merge_amount);
}
}

```

We can see the final result needs a merge, and it is a tiny amount of dust 2.

```

running 1 test
strip_and_merge_amount:
  .strip_sy_amount: 29
  .merged_base_amount: 33
  strip_base_amount: 31
diff_merge_amount: 2
test state::orderbook::tests::virtual_virtual_single_fill_rounding_diff ... ok

```

Resolution

This issue has been fixed by [acbc61e](#) and [0246e3c](#).

EXPONENT_ORDERBOOK

[I-01] Missing check for threshold_amount during configuration

Identified in commit [8c8b2db](#).

In the `validate_accounts` function of `post_offer`, the check for `post_amount` relies on `threshold_amount.checked_add(1)`. If `threshold_amount` equals `u64::MAX`, `checked_add(1)` returns `None`, and the subsequent `unwrap()` panics, causing the transaction to fail. The panic is thrown before the intended validation and results in an incorrect error message being returned.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/post_offer.rs */
186 | post_amount
187 |     .checked_sub(
188 |         raw.configuration_options
189 |             .threshold_amount
190 |             .checked_add(1)
191 |             .unwrap(),
192 |     )
193 |     .expect("Post amount must be greater than threshold amount");
```

During `initialize_orderbook` and `set_configuration_options`, no corresponding check is performed to constrain the allowable range of `threshold_amount`. As a result, if the `admin_signer` attempts to use `u64::MAX` to represent an unbounded `post_offer` constraint, the operation fails unexpectedly.

It is recommended to enforce that `threshold_amount` is not equal to `u64::MAX` during initialization.

Resolution

The team acknowledged this finding.

EXPONENT_ORDERBOOK

[I-02] Missing offer expiry time check

Identified in commit [8c8b2db](#).

During the `post_offer` instruction, the user sets an `expiry_seconds` value for the offer. The offer's expiration time is computed as the current timestamp plus this `expiry_seconds`.

```
/* solana/programs/exponent_orderbook/src/state/offer.rs */
036 | pub fn new(
037 |     virtual_offer: bool,
038 |     offer_type_flag: u8,
039 |     amount: u64,
040 |     expiry_seconds: u32,
041 |     offer_options: OfferOptions,
042 | ) -> Self {
043 |     let now = now();
044 |     let expiry_at = now
045 |         .checked_add(expiry_seconds)
046 |         .expect("Expiry time overflow");
047 |     let fill_or_kill = match offer_options {
048 |         OfferOptions::FillOrKill(flag) => flag,
049 |     };
```

Since `YT` tokens become worthless after the core vault reaches maturity, any offer cannot be executed once the vault has matured. To prevent offers from remaining valid beyond the point where execution is impossible, it is recommended to enforce a check ensuring that the offer's expiration time does not exceed the core vault's maturity timestamp.

Resolution

This issue has been fixed by [4fe96b6](#).

EXPONENT_ORDERBOOK

[I-03] Zero filled amount leads to maker loss and vault SY depletion

Identified in commit [8c8b2db](#).

In `post_offer` or `market_offer` function, after computing `available_amount`, `quoted_amount`, `taken_amount` and `filled`, the code unconditionally increments the filled counter and pushes an `OfferFillParams` entry without verifying that `taken_amount > 0` and `filled > 0`.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
469 | let available_amount = counter_offer.get_available_amount(
470 |     counter_offer_price,
471 |     sec_remaining,
472 |     sy_exchange_rate_float,
473 |     pt_price_asset_optional,
474 | );
476 | // Compute quote based on counter offer price
477 | let quoted_amount = compute_quote_amount(
478 |     offer_type,
479 |     remaining_amount,
480 |     counter_offer_price,
481 |     sy_exchange_rate_float,
482 |     sec_remaining,
483 |     virtual_offer,
484 |     pt_price_asset_optional,
485 | )?;
487 | let taken_amount = if available_amount > quoted_amount {
488 |     quoted_amount
489 | } else {
490 |     available_amount
491 | };
493 | // Compute filled into counter offer amount without virtualization
494 | let filled = compute_quote_amount_no_virtual(
495 |     counter_offer.get_type(),
496 |     taken_amount,
497 |     counter_offer_price,
498 |     sy_exchange_rate_float,
499 |     sec_remaining,
500 |     pt_price_asset_optional,
501 | )?;
```

Since the computation of `filled` applies a round down operation, and `yt_to_sy_rate` gradually approaches zero over time, it becomes possible in the scenario where `taker = BuyYt` and `counter_offer = SellYt` for `taken_amount > 0` while `filled == 0`.

```
pt_price_asset = Exp(ln(1 + apy)*(-seconds_remaining/seconds_in_year))
yt_to_sy_rate = (1 - pt_price_asset) / sy_exchange_rate
filled[SY] = floor(taken_YT * yt_to_sy_rate)
```

Case 1: counter_offer is non-virtual

In this case, `fill_offer` deducts the positive `taken_amount` from the counter_offer's posted amount, while the taker's `remaining_amount` is reduced by `filled`. Thus, the taker does not actually lose any funds, and the received amount computed from `filled_amount` is also zero for the taker.

Meanwhile, on the `counter_offer` side, the YT balance is reduced by `taken_amount`, even though no actual value is transferred to the taker.

Thus, in this process, the taker receives nothing, the counteroffer receives nothing, yet the counteroffer's YT is partially evaporated due to the floor-rounded conversion.

Taker side:

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
954 | let taker_fill_escrow_result = self.get_amounts_on_filling_escrow(
955 |     amount_sended,
956 |     filled_amount,
957 |     sy_exchange_rate,
958 |     offer_type.counter_type(),
959 |     incoming_virtual_offer,
960 |     true,
961 |     &fee_rates,
962 | );

/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
527 | // Update remaining amount after filling the counter offer with virtual or non-virtual terms
528 | remaining_amount = calculate_remaining_amount_on_counter_offer_filling(
529 |     virtual_offer,
530 |     offer_type,
531 |     remaining_amount,
532 |     filled,
533 |     taken_amount,
534 |     sy_exchange_rate_float,
535 | );
```

Counteroffer side:

```
/* solana/programs/exponent_orderbook/src/state/offer.rs */
153 | } else {
154 |     offer.amount = offer.amount.checked_sub(taken_amount).unwrap();
155 |
156 |     StripAndMergeAmount {
157 |         strip_sy_amount: 0,
158 |         merged_base_amount: 0,
159 |         taker_fee_amount: 0,
160 |         maker_fee_amount: 0,
161 |     }
162 | }
```

Case 2: counter_offer is virtual

When the `counter_offer` is virtual, `filled == 0`, but `taken_amount_sy = floor(taken_amount/sy_exchang`

`e_rate) > 0`. Thus, even with `filled == 0`, the system strips SY from the orderbook vault to PT and YT, causing the available SY in the pool to decrease unexpectedly.

```

/* solana/programs/exponent_orderbook/src/state/offer.rs */
137 | // total available amount can be taken = sy_amount (self.amount ) + sy (from buy_yt offer)
138 | let taken_amount_sy = calculate_sy_amount_from_base(taken_amount, sy_exchange_rate);
139 | offer.amount = offer
140 |     .amount
141 |     .checked_add(filled_amount)
142 |     .unwrap()
143 |     .checked_sub(taken_amount_sy) // cause taken YT to SY after merge
144 |     .unwrap();
145 |
146 | StripAndMergeAmount {
147 |     strip_sy_amount: taken_amount_sy,
148 |     merged_base_amount: 0,
149 |     taker_fee_amount: 0,
150 |     maker_fee_amount: 0,
151 | }
152 | }

```

Since `yt_to_sy_rate` approaches zero as time passes, the likelihood of `taken_amount > 0` but `filled == 0` becomes progressively higher, increasing the probability of the incorrect burn or SY loss for orderbook vault described above.

It is recommended to require both `taken_amount > 0` and `filled > 0`, otherwise skip the offer and continue scanning. Optionally, also skip offers where `available_amount == 0` or `offer.amount <= threshold_amount` to reduce zero-fills.

Resolution

This issue has been fixed by [32aa0a6](#) and [8cc2cbe](#).

EXPONENT_ORDERBOOK

[I-04] Nearest price rounding violates user-defined limit

Identified in commit [8c8b2db](#).

The rounding logic in `get_modified_price_decimals` works by rounding an input APY to the configured `price_decimals` using nearest rounding, and then converting the result to base points. This derived value is used in `post_offer` and `market_offer` as the taker's price key.

However, nearest rounding can exceed user limits, rounding up for `BuyYt` may result in a higher price, and rounding down for `SellYt` may result in a lower one.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
1272 | pub fn get_modified_price_decimals(&self, price_apy: f64) -> u32 {
1273 |     let decimals = self.configuration_options.price_decimals as i32;
1274 |     let factor = 10_f64.powi(decimals);
1275 |     let rounded_price = (price_apy * factor).round() / factor;
1276 |
1277 |     (rounded_price * PRICE_BASE_POINTS as f64).round() as u32
1278 | }
1279 |
```

It is recommended to apply direction-aware rounding, round down for `BuyYt` and up for `SellYt`.

Resolution

This issue has been fixed by [03c78af](#).

EXPONENT_ORDERBOOK

[I-05] Missing validation for fee_rate_bp

Identified in commit [8c8b2db](#).

In the orderbook protocol's fee calculation, the effective fee rate is computed as: `ceil( exp(ln_fee_rate * normalized_sec_remaining) - 1 )`. where `normalized_sec_remaining` is `(orderbook.expiration_ts - now) / ONE_YEAR`.

```
/* solana/libraries/time_curve/src/math.rs */
022 | /// Calculate the fee rate for a given time remaining and fee rate root
023 | pub fn fee_rate<N: Num>(ln_fee_rate_root: N, sec_remaining: u64) -> N {
024 |     let normalized_sec_remaining = normalized_sec_remaining::<N>(sec_remaining);
025 |     let ln_fee_rate = ln_fee_rate_root * normalized_sec_remaining;
026 |     ln_fee_rate.exp()
027 | }
```

Since there is currently no validation on `ln_fee_rate` during initialization or configuration, the expression `ceil(exp(ln_fee_rate * normalized_sec_remaining) - 1)` may produce a value greater than 1, which would cause trades to fail.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/set_configuration_options.rs */
055 | SetConfigurationOptionsEnum::TakerFeeRateBp(taker_fee_rate_bp) => {
056 |     orderbook_raw.configuration_options.ln_taker_fee_rate = taker_fee_rate_bp;
057 | },
058 | SetConfigurationOptionsEnum::MakerFeeRateBp(maker_fee_rate_bp) => {
059 |     orderbook_raw.configuration_options.ln_maker_fee_rate = maker_fee_rate_bp;
060 | },
```

It is recommended to add validation for `ln_fee_rate` during initialization and configuration to prevent unreasonable or invalid fee settings.

Resolution

This issue has been fixed by [7c14065](#).

EXPONENT_ORDERBOOK

[I-06] Orderbook rent exhaustion DoS risk

Identified in commit [8c8b2db](#).

During orderbook trades involving **YT** tokens (or when staging **YT** yield), the orderbook program deposits to or withdraws from the core program vault's **YT user_yield_position** via CPI.

```
/* solana/programs/exponent_core/src/instructions/vault/deposit_yt.rs */
021 | pub struct DepositYt<'info> {
022 |     /// Permissionless depositor - not necessarily the owner of the YieldTokenPosition
023 |     #[account(mut)]
024 |     pub depositor: Signer<'info>,
025 |
026 |     /// Vault that manages YT
027 |     #[account(
028 |         mut,
029 |         has_one = sy_program,
030 |         has_one = escrow_yt,
031 |         has_one = address_lookup_table,
032 |         has_one = yield_position
033 |     )]
034 |     pub vault: Account<'info, Vault>,
035 |
036 |     /// Position data for YT deposits, linked to vault
037 |     #[account(
038 |         mut,
039 |         has_one = vault,
040 |         realloc = YieldTokenPosition::size_of(vault.emissions.len()),
041 |         realloc::payer = depositor,
042 |         realloc::zero = false,
043 |         // no duplicate YT positions
044 |         constraint = user_yield_position.key() != yield_position.key()
045 |     )]
```

In the core program, the **YT** deposit/withdraw and stage yield related instructions may reallocate the **user_yield_position** account when new emissions are added to the vault. This reallocation requires the instruction signer to pay the additional rent required for the increased account size. In the orderbook CPI context, the payer effectively becomes the **orderbook** account.

```
/* solana/programs/exponent_orderbook/src/state/orderbook.rs */
336 | impl RawOrderbook {
337 |     pub fn signer_seeds(&self) -> [Vec<u8>; 4] {
338 |         let signer_seeds_vec: [Vec<u8>; 4] = [
339 |             ORDERBOOK,
340 |             &self.vault.as_ref(),
341 |             &self.seed_id,
342 |             &self.signer_bump,
343 |         ]
344 |         .map(|s| s.to_vec());
345 |
```

```

346 |     signer_seeds_vec
347 | }

```

However, the `orderbook` account only reserves rent during initialization and does not collect additional SOL afterward. When the underlying vault adds a new emission, subsequent `orderbook` YT trade or stage yield operations can trigger a reallocation in the core program, requiring the orderbook account to pay extra rent. This may reduce the orderbook account's lamports below the rent-exempt minimum.

```

/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/realloc_orderbook_account.rs */
032 | pub fn make_orderbook_account(ctx: &Context<ReallocOrderbookAccount>, seed_id: u32) -> Result<()> {
033 |     let orderbook_account = &ctx.accounts.orderbook;
034 |     let rent = &ctx.accounts.rent;
035 |     let vault = &ctx.accounts.vault;
036 |     let payer = &ctx.accounts.payer;
037 |
038 |     let orderbook_space: u64 = ORDERBOOK_SIZE as u64;
039 |     let lamports = rent.minimum_balance(ORDERBOOK_SIZE);

```

Since the Solana runtime prevents transactions that would reduce an account below the required minimum balance, this creates a DoS risk; these instructions can fail once the orderbook account no longer has sufficient lamports to cover the reallocation rent.

It is recommended to add explicit rent-fee funding requirements in the orderbook instructions that trade YT tokens or stage YT yield.

Resolution

This issue has been fixed by [5c18533](#).

EXPONENT_ORDERBOOK

[I-07] DoS the orderbook with no penalty

Identified in commit [8c8b2db](#).

Currently, any user can place orders on the orderbook, but offers can only be removed by the offer creator or by an administrator after expiration. The orderbook does not enforce restrictions on price or expiry duration, and unfilled offers are fully refunded. This allows malicious users to spam offers with unreasonable prices and near-infinite expiration times without any penalty (aside from the capital lockup, depending on [threshold_amount](#)).

And, since each orderbook has hard capacity limits: [100](#) distinct prices, [250](#) distinct users, and [500](#) total offers, a malicious user can easily occupy all 100 price slots, causing the orderbook to be unusable and a DoS.

Reference implementations such as OpenBook V2 address this by evicting offers with the most extreme prices when the capacity is reached to maintain operation. While this might disadvantage makers in crowded markets by canceling their orders, it prevents the system from locking up. If eviction is not desired, we recommend at least increasing the cost for placing unreasonable offers to ensure a DoS attack cannot be performed without a significant penalty.

Resolution

This issue has been fixed by [a9a855e](#).

EXPONENT_ORDERBOOK

[I-08] Missing explicit owner check in set_configuration_options

Identified in commit [8c8b2db](#).

The `set_configuration_options` instruction validates the `orderbook` discriminator but does not explicitly verify the account owner.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/set_configuration_options.rs */
016 | impl<'i> SetConfigurationOptions<'i> {
017 |     fn validate(&self) -> Result<()> {
018 |         self.validate_admin()?;
019 |         Ok(())
020 |     }

022 |     fn validate_admin(&self) -> Result<()> {
023 |         let data = &mut self.orderbook.data.borrow_mut();
024 |
025 |         let orderbook_raw: &mut RawOrderbook = bytemuck::from_bytes_mut(&mut data[8..]);
026 |
027 |         require!(
028 |             self.admin_signer.key() == orderbook_raw.admin,
029 |             OrderbookValidationsErrors::WrongAdmin
030 |         );
031 |
032 |         Ok(())
033 |     }

035 |     pub fn set_configuration_options(
036 |         &self,
037 |         set_configuration_options_enum: SetConfigurationOptionsEnum,
038 |     ) -> Result<()> {
039 |         let data = &mut self.orderbook.data.borrow_mut();
040 |
041 |         require!(
042 |             &data[..8] == OrderbookMeta::DISCRIMINATOR,
043 |             ExponentOrderbookError::ExponentOrderbookDiscriminatorMismatch
044 |         );
046 |         let orderbook_raw: &mut RawOrderbook = bytemuck::from_bytes_mut(&mut data[8..]);
064 |     }
065 | }
```

Since this instruction modifies the `orderbook` data, the runtime will automatically revert the transaction if the account is not owned by the executing program, so there is no security risk. However, we still recommend adding an explicit owner check as the other instructions do.

Resolution

This issue has been fixed by [715daa4](#).

EXPONENT_ORDERBOOK

[Q-01] Missing user distribution for emission rewards

Identified in commit [8c8b2db](#).

In the core program, users who deposit **YT** tokens receive both **SY** interest and emission rewards. In the orderbook protocol, users who own **YT** are only credited for the corresponding **SY** interest through the `s tage_yt_interest` logic. The protocol does not calculate or distribute emission rewards to users.

Instead, the orderbook admin can collect all emission rewards associated with **YT** held inside the orderbook via the `CollectAdminEmission` instruction. Users themselves have no ability to claim these rewards.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/collect_admin_emission.rs */
056 | pub fn handler<'info>(<
057 |     ctx: Context<'_, '_, '_, 'info, CollectAdminEmission<'info>>,
058 |     amount: Amount,
059 |     index: u16,
060 | ) -> Result<CollectAdminEmissionEvent> {
061 |     let signer_seeds_vec = {
062 |         let mut data = ctx.accounts.orderbook.data.borrow_mut();
063 |         let orderbook_raw: &mut RawOrderbook = bytemuck::from_bytes_mut(&mut data[8..]);
064 |
065 |         orderbook_raw.signer_seeds()
066 |     };
067 |
068 |     let signer_seeds = signer_seeds_vec_to_slice(&signer_seeds_vec);
069 |
070 |     let collect_emission_data = do_cpi_collect_emission(
071 |         ctx.remaining_accounts,
072 |         *ctx.accounts.exponent_core.key,
073 |         index,
074 |         amount,
075 |         &[&signer_seeds],
076 |     )?;
077 | }
```

Given this design, is there any mechanism that ensures the orderbook admin will redistribute these emission rewards fairly to users?

Resolution

The team acknowledged this finding and clarified that this is the intended behavior.

EXPONENT_ORDERBOOK

[Q-02] Discrepancies between event data and the actual transferred amount

Identified in commit [8c8b2db](#).

There are multiple instructions in the program where the `amount` recorded in events and return data does not match the actual amount of tokens transferred. We want to confirm if this behavior is expected.

1. The amount in the event is not the actual transfer-in amount

In `post_offer` and `market_offer`, the `amount_in` or `amount` fields in the events reflect the `amount` provided in the user parameters (the intended consumption). However, the actual consumed amount corresponds to the quantity executed in the `transfer`.

The name `amount_in` in the `post_offer` event suggests it should represent the actual amount deposited into the orderbook.

```
/* solana/programs/exponent_orderbook/src/instructions/orderbook/post_offer.rs */
245 | pub fn handler<'info>(<
248 |     amount: u64,
253 | ) -> Result<PostOfferEvent> {
309 |     // Transfer funds
310 |     ctx.accounts.do_transfers(
311 |         amount
312 |         .checked_sub(post_offer_output.remaining_amount)
313 |         .unwrap(),
318 |     );
343 |     let event = PostOfferEvent {
348 |         // @audit: actual transfer in amount is `amount-post_offer_output.remaining_amount`
348 |         amount_in: amount,
360 |     };
361 |
362 |     emit_cpi!(event);
363 |
364 |     Ok(event)
365 | }

/* solana/programs/exponent_orderbook/src/instructions/orderbook/market_offer.rs */
244 | pub fn handler<'info>(<
247 |     amount: u64,
251 | ) -> Result<MarketOfferEvent> {
306 |     // Transfer funds
307 |     ctx.accounts.do_transfers(
308 |         amount
309 |         .checked_sub(post_offer_output.remaining_amount)
310 |         .unwrap(),
315 |     );
340 |     let event = MarketOfferEvent {
344 |         amount,
354 |     };
355 | }
```

```

356 |     emit_cpi!(event);
357 |
358 |     Ok(event)
359 | }

```

2. The amount in the event is not the actual transfer-out amount

After expiration, YT tokens are valueless, so the orderbook skips the actual token transfer for YT withdrawal requests. However, the `amount_yt_out` in the event remains non-zero. This might confuse programs that rely on return data to verify transferred token amounts.

The affected instructions include `withdraw_funds`, `remove_offer`, and `collect_admin_fee`.

```

/* solana/programs/exponent_orderbook/src/instructions/orderbook/withdraw_funds.rs */
261 | pub fn handler<'info>(<
266 | ) -> Result<WithdrawFundsEvent> {
301 |     if !is_expired {
315 |         ctx.accounts
316 |             .do_transfer_yt_out(yt_amount, &[&signer_seeds])
317 |             .unwrap();
318 |     }
328 |     let event = WithdrawFundsEvent {
336 |         amount_yt_out: yt_amount,
337 |     };
338 |
339 |     emit_cpi!(event);
340 |
341 |     Ok(event)
342 | }

/* solana/programs/exponent_orderbook/src/instructions/orderbook/remove_offer.rs */
134 | fn do_transfers(
140 | ) -> Result<()> {
151 |     if !is_expired {
162 |         self.do_transfer_yt_out(offer.amount, seeds).unwrap();
163 |     }
168 | }

235 | pub fn handler<'info>(<
238 | ) -> Result<RemoveOfferEvent> {
266 |     ctx.accounts
267 |         .do_transfers(ctx.remaining_accounts, &[&signer_seeds], offer, is_expired)?;
268 |
269 |     let event = RemoveOfferEvent {
276 |         amount_out: offer.amount,
278 |     };
279 |
280 |     emit_cpi!(event);
281 |
282 |     Ok(event)
283 | }

/* solana/programs/exponent_orderbook/src/instructions/orderbook/admin/collect_admin_fee.rs */
224 | pub fn handler<'info>(<
226 | ) -> Result<CollectAdminFeeEvent> {

```

```
273 |     if !is_expired {  
286 |         ctx.accounts  
287 |             .do_transfer_yt_out(yt_amount, &[&signer_seeds])  
288 |             .unwrap();  
289 |     }  
302 |  
303 |     let event = CollectAdminFeeEvent {  
306 |         yt_amount,  
309 |     };  
310 |  
311 |     emit_cpi!(event);  
312 |  
313 |     Ok(event)  
314 | }
```

Resolution

This issue has been fixed by [b6c4c4e](#) and [12ebe9a](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Exponent Labs Ltd. (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

